Constant Buffers

Direct3D – Constant buffers

- Similar to vertex and index buffers
  - Another ID3D11Buffer
  - Holds numbers in GPU memory
- Contain numeric data for *global shader variables*
  - Hence "**Buffer**"
- Global shader variables
  - Shader data that can be set *externally* (C++)
  - However, shader itself cannot change these values
  - Hence "**Constant**"

Constant Buffer on GPU

(Holds numbers)

Vertex Buffer on GPU

(Also holds numbers)

Shaders – cbuffer

- Stands for "Constant Buffer"
- A construct within a shader file
- Does three things:
  - Declares **global shader variables**
  - Describes **variable layout** in memory
  - Defines the slot (**register**) to which a corresponding D3D Constant Buffer should be bound

Shader code with cbuffer

```
cbuffer PerObjectData : register(b0)
{
    float4x4 worldMatrix;
    float4   colorTint;
}

// Rest of shader code here
// (structs, main(), etc.)
```

Constant Buffers vs. cbuffers

- Constant Buffer holds numbers
  - In a particular order
- cbuffers define variables
  - In a particular order
- They must match!
  - If cbuffer wants 20 floats
  - Buffer needs 20 floats
  - In the same order

Constant Buffer on GPU

Bound to cbuffer register 0

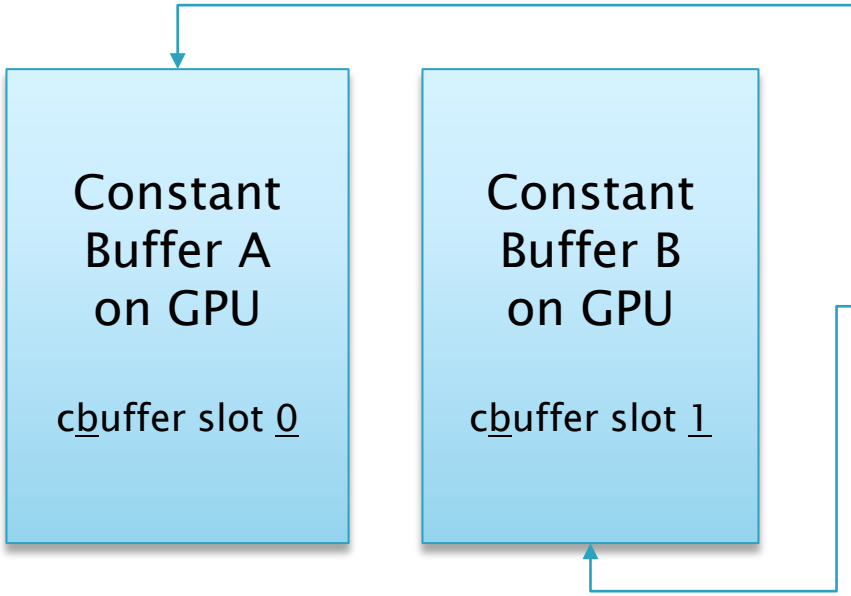
Shader code with constant buffers

```
cbuffer PerObjectData : register(b0)
{
    float4x4 worldMatrix;
    float4   colorTint;
}

// Rest of shader code here
// (structs, main(), etc.)
```

Why are there multiple "slots"?

- It's possible to use more than one at a time
  - An advanced optimization
  - We'll just use one for now



Shader code with constant buffers

```
cbuffer PerObjectData : register(b0)
{
    float4x4 worldMatrix;
    float4   colorTint;
}

cbuffer PerFrameData : register(b1)
{
    float4x4 viewMatrix;
    float4x4 projMatrix;
}

// Rest of shader code here
// (structs, main(), etc.)
```

Stats on cbuffers

- Each can hold up to 4096 vectors
  - One vector can hold between one and four 32-bit values
  - (A one component vector is just a scalar)
  - (A matrix is just four 4-component vectors)
  - We won't use anywhere near this many
- Can bind up to 14 constant buffers per shader stage
  - Shader stages are vertex, pixel, etc.
  - We won't use anywhere near this many

Defining a cbuffer

Identifier – Name it anything you'd like

Register – Defines which "slot" the buffer should be bound to

```
cbuffer externalData : register( b0 )
{
    matrix world;
    matrix view;
    matrix projection;
};
```

b – Signifies a constant buffer register. Cbuffers always use "b"

0 – The first register slot

Why though?

- Can't I just copy data one variable at a time? No.
  - Copying data to the GPU has overhead
  - Better to copy a large chunk of data at once
  - Than to copy small chunks over and over

- Constant buffers let us update entire groups of variables
  - The *only* way to update global shader variables in Direct3D

C++ (CPU)

```
struct PerObject
{
    XMFLOAT4X4 worldMatrix;
    XMFLOAT4 colorTint;
};
```

Fill & copy

HLSL (GPU)

```
cbuffer PerObject : register(b0)
{
    float4x4 worldMatrix;
    float4 colorTint;
}
```

Creating D3D constant buffers in C++

- Another ID3D11Buffer*
- Size should match shader's cbuffer definition
- Bind flag: D3D11_BIND_CONSTANT_BUFFER
- Does not need initial data

How many buffers do I need?

- One C++ buffer per cbuffer in *each* shader
- If you had:
  - **2** cbuffers in your vertex shader
  - **1** cbuffer in your pixel shader
- You'd need **three** ID3D11Buffer objects
  - (At maximum – more on optimizations later)

Matching ID3D11Buffers & cbuffers

- How do we match:
  - The variables in a *shader's cbuffer*
  - The raw data in an *ID3D11Buffer*
- Shader's cbuffer layout is apparent
  - You wrote it in the shader
- ID3D11Buffer is just a raw buffer of bytes
  - It can hold any data we want

Constant buffer data in C++

- Create one or more matching structs in C++
  - Each should match the layout of a cbuffer
- In C++
  - Create one of these structs
  - Fill it with data
- Then copy the *entire C++ struct* to the ID3D11Buffer
  - This is a CPU → GPU copy
  - This ensures your data layouts match

Copying data to the GPU – Two approaches

- UpdateSubresource()
  - Works on any buffer
  - Can copy (overwrite) part or all of a buffer
  - Incurs extra CPU overhead* – Not ideal for frequent copies (every frame)

- Map() / memcpy() / Unmap()
  - Only works on buffers specifically created as “dynamic”
  - Locks the buffer to prevent GPU use
  - Copy data using memcpy, then unmap (unlock) immediately
  - Better choice for constant buffers

- *See GDC 2005 talk by John McDonald for more efficiency details

Activating (binding) constant buffers

- Similar to binding other buffers and shaders
  - Must specify index and how many you're setting at once

```
context->VSSetConstantBuffers()
context->PSSetConstantBuffers()
```

- 3 Parameters:
  - **StartSlot** – The index you're setting
  - **NumBuffers** – # of buffers you're setting with this call
  - **ID3D11Buffer**** – Array of buffer pointers (or address of single pointer)

Bind one ID3D11Buffer per cbuffer

- If Vertex Shader has 2 cbuffers
  - `cbuffer PerObject : register( b0 ) { … }`
  - `cbuffer PerFrame  : register( b1 ) { … }`

- Activate two buffers in C++
  - `VSSetConstantBuffers(0, 1, perObjectBuffer.GetAddressOf());`
  - `VSSetConstantBuffers(1, 1, perFrameBuffer.GetAddressOf());`

    -or-
  - `VSSetConstantBuffers(0, 2, arrayOfBuffers);`

Multiple constant buffers?

- Why not put everything into one cbuffer?
  - You technically could
  - And we will for now
- But a single constant buffer is not always optimal
- You have to update the entire buffer when *any* data changes
  - Object moves?
  - Copy the entire

Example: Multiple constant buffers

- World matrix will be different for each object
- But View & projection camera matrices?
  - Usually change at most once per frame
  - Must we re-send view & projection to GPU for each object?
  - Ideally no!
- **Optimization**: Organize external data into buffers based on update frequency

Common cbuffer organization

- Per-Object Buffer
  - World matrix, raw position, etc.
  - Data that differs between *objects*
- Per-Material Buffer
  - Texture scaling, color tint, etc.
  - Data that differs between *surface materials*
- Per-Frame Buffer
  - Lights, view matrix, projection matrix, time, etc.
  - All objects drawn this frame will *share* this data
  - Copy this data to the GPU only once each frame!

Constant buffer data packing

- HLSL compiler might *alter* your cbuffer layout
- Packing Rule:
  - Cbuffer vectors *cannot* cross 16-byte boundaries
- If a single vector does cross that boundary
  - Compiler will pad out the cbuffer to “fix” this
  - Compiler puts “extra space” between variables

Data packing example – OK

```
cbuffer Simple
{
      float4 color;              // Nothing crosses 16-byte boundary
      float4 anotherColor;       // Another 16 bytes
      float2 uvCoord;            // 8 bytes
};
```

- This one is fine
  - No extra packing will occur

Data packing example – OK

```
cbuffer AlsoSimple
{
      float time;          // 4 bytes
      float3 position;     // 12 bytes – Still good
      float2 uvCoord;      // Starts after boundary!
};
```

- Also fine
  - No extra packing will occur

Data packing example – Rule violation

```
cbuffer NotSimple
{
      float2 uvCoord;      // Doesn't cross boundary
      float4 color;        // DOES cross the boundary!

      // It will actually start 16 bytes into the buffer!

      // The compiler effectively puts an extra float2 between the
      // existing variables to "fix" the issue – without telling you!
};
```

- This one will be altered!
  - Your C++ structure layout MUST take this into account!
  - Or the data won't line up correctly

More like constant BUMMER

- This constant buffer stuff seems complicated
- And it kinda is
- The tradeoff is efficiency
  - (If you use them correctly)